

INSTITUTO FEDERAL DO ESPÍRITO SANTO — CAMPUS SERRA

Bacharelado em Sistemas de Informação

Técnicas de Programação Avançada

Trabalho 1 — Análise de complexidade em estruturas de listas

Relatório técnico

Integrantes do grupo

Kaiva Gums

Caio Rangel

João Victor Araujo

Professor: Victorio Albani

Serra — ES, setembro de 2026

1. Desenvolvimento da biblioteca e do programa de teste

1.1 Visão geral da solução

O trabalho consiste em uma biblioteca de lista encadeada genérica, `ListaEncadeada<T>`, e um programa de teste que a utiliza para gerenciar um cadastro de alunos. A classe recebe no construtor um `Comparator<T>` e um booleano que define se a lista opera em regime ordenado ou não-ordenado, e implementa a interface `IColecao<T>`, que expõe apenas adicionar, pesquisar, remover e `quantidadeNos` -- o programa de teste manipula a lista somente por esse contrato, sem acessar `No<T>` ou os campos internos diretamente.

Como domínio, o grupo optou por `Aluno` (matrícula, nome e nota) em vez do `Contato` (nome e telefone) sugerido no enunciado, seguindo o exemplo disponível no repositório do professor. A troca não altera a `ListaEncadeada`: a estrutura é genérica e desconhece a classe `Aluno`; mudam apenas a classe de domínio e os dois comparadores utilizados (`ComparadorAlunoPorMatricula` e `ComparadorAlunoPorNome`), que cumprem o mesmo papel que `telefone` e `nome` cumpriram no domínio original uma chave numérica única e uma chave textual sujeita a repetição.

1.2 Atuação de cada integrante

Integrante	Matrícula	Atividades desempenhadas
Kaiva Gums	20241BSI0130	Criação do GitHub, desenvolvimento de código, desenvolvimento de relatório
Caio Rangel	20212BSI0022	Desenvolvimento do Código, desenvolvimento de relatório
João Victor Araujo	20241BSI0319	Desenvolvimento de Código, teste das funcionalidades, desenvolvimento de relatório

1.3 Uso de ferramentas de inteligência artificial

Ferramenta	Em que etapa foi usada	O que foi aceito, ajustado ou descartado
Claude Code	Geração do Script de tests	O código base, com algumas modificações em como e o que contar na hora de registrar os testes
Claude Code	Checagem da corretude do código	Criticou o estado do código e sugeriu o que devia ser corrigido. Implementamos as mudanças à mão (linha a linha)

1.4 Repositório e execução

Endereço do repositório: https://github.com/sKaivar/Complexidade_de_Estruturas-TPA

Estrutura do projeto:

- dados/ – Resultados do Script de carga
- relatorio/ – Resultados do Script de medição
- src/scripts/ – Scripts de auxílio ao programa
 - src/scripts/GeradorArquivos.java – geração dos arquivos de carga
 - src/scripts/Benchmark.java – bateria de medições (CSV)
- src/colecao – Interface
 - src/colecao/IColecao.java – contrato da coleção
- src/ dominio/ – Main e classes Aluno e de comparação
 - src/ dominio/Aluno.java – classe de domínio
 - src/ dominio/Comparador*.java – critérios de comparação
 - dominio/ComparadorAlunoPorMatricula – Compara por matrícula
 - dominio/ComparadorAlunoPorNome – Compara por nome
 - src/ dominio/Main.java – programa de teste (menu)
- relatorio/ – Resultados do Script de medição
- src/listaencadeada/ – Arquivos de Lista e Nós
 - src/listaencadeada/No.java – nó da lista
 - src/listaencadeada/ListaEncadeada.java – biblioteca de Lista

Compilação e execução:

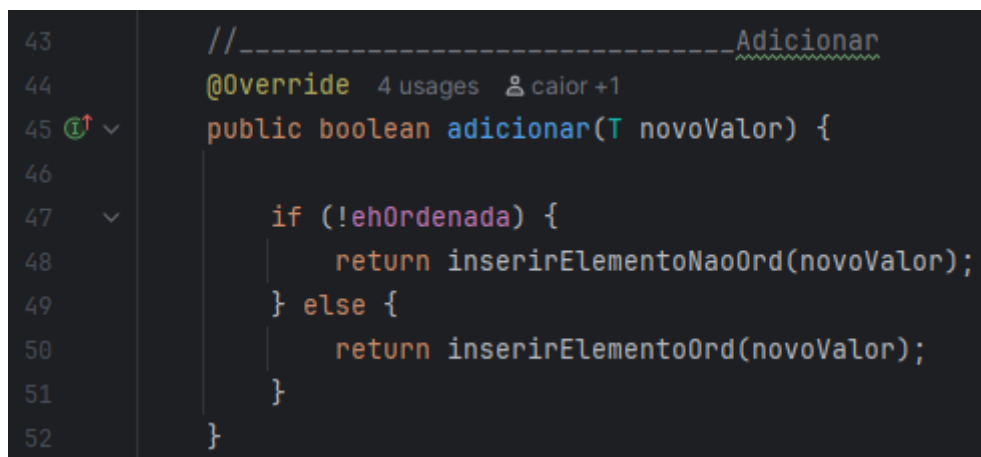
```
javac -encoding UTF-8 -d out $(find src -name "*.java")
java -cp out scripts.GeradorArquivos 25000 50000 100000 200000
java -cp out dominio.Main
java -cp out scripts.Benchmark
```

2. Análise matemática de complexidade

Todos os testes foram feitos em uma máquina que possui um Intel i5-10400F 2.90GHz e 16Gb DDR4. Foram realizadas cinco repetições do teste. Onde o primeiro conjunto de dados foi descartado, enquanto os outros quatros foram utilizados para fazer uma média dos valores que eles retornaram. Cada teste compõe de N repetições. No nosso caso foram feitas 11 repetições para cada teste, e foi tirada uma mediana dessas 11 repetições.

Com isso, as próximas seções irão dissertar sobre a matemática de complexidade de cada método desenvolvido pela equipe. Nela iremos identificar e interpretar as expectativas de tempo de resposta de cada método.

2.1 adicionar



```
43 //-----Adicionar
44 @Override 4 usages 3 calor +1
45 public boolean adicionar(T novoValor) {
46
47     if (!ehOrdenada) {
48         return inserirElementoNaoOrd(novoValor);
49     } else {
50         return inserirElementoOrd(novoValor);
51     }
52 }
```

Figura 2.1.1

```
public boolean adicionar(T novoValor) {

    if (!ehOrdenada) {
        return inserirElementoNaoOrd(novoValor);
    } else {
        return inserirElementoOrd(novoValor);
    }
}
```

Método de adicionar elemento, dependendo do tipo valor de *ehOrdenado* ele chama um dos dois métodos a seguir. Considerando apenas isso, a complexidade dele é $O(1)$, já que o if e o else vão ser executados apenas uma vez.

```

54  ✓      public boolean inserirElementoNaoOrd(T novoValor) { 1 usage  🧑 caior
55          No<T> novoNo = new No<>(novoValor);
56
57  ✓      if (prim == null) {
58          prim = novoNo;
59          ult = novoNo;
60      }else{
61          novoNo.setProx(prim);
62          prim = novoNo;
63      }
64
65          quant++;
66          return true;
67      }

```

Figura 2.1.2

```

public boolean inserirElementoNaoOrd(T novoValor) {
    No<T> novoNo = new No<>(novoValor);

    if (prim == null) {
        prim = novoNo;
        ult = novoNo;
    }else{
        novoNo.setProx(prim);
        prim = novoNo;
    }

    quant++;
    return true;
}

```

Caso seja não-ordenado, essa é a lógica que ele vai aplicar, colocando sempre o novo elemento em primeiro lugar. E sua complexidade também será $O(1)$, já que vai ser executado apenas uma vez também, no momento que é chamada para adicionar um elemento na lista. E como não será feita nenhuma análise nela, o elemento vai ser apenas inserido na lista.

```

60 public boolean inserirElementoOrd(T novoValor){ Usage: <word>1
61
62     No<T> novoNo = new No<>(novoValor);
63     No<T> atual, ant;
64
65     atual = this.prim;
66     ant = null;
67
68     if(this.prim == null){//Se lista estiver vazia, cria novo no e adiciona no prim (primeiro na lista)
69         this.prim = this.ult=novoNo;
70     }else{// Enquanto não é o ultimo da lista e é maior que o atual, vai para o próximo
71
72         while (atual != null && comparador.compare(novoValor, atual.getValor()) >= 0) { // Enquanto atual não for null e novoValor for maior ou igual que atual.getValor()
73             ant = atual;// No atual vira o anterior
74             atual = atual.getProx();// Atual passa apontar para o próximo No
75         }
76
77         if(ant == null){// No for menor que o primeiro No da lista, logo tem que ser o novo primeiro No
78             novoNo.setProx(this.prim);
79             this.prim = novoNo;
80
81         } else if (atual == null ) { // Chegou ao final da lista, No vai ser inserido na última posição
82             this.ult.setProx(novoNo);
83             this.ult = novoNo;
84
85         } else{
86             ant.setProx(novoNo);
87             novoNo.setProx(atual);
88         }
89     }
90
91     this.quant++;
92     return true;
93 }
94
95

```

Figura 2.1.3

```

public boolean inserirElementoOrd(T novoValor){

    No<T> novoNo = new No<>(novoValor);
    No<T> atual, ant;

    atual = this.prim;
    ant = null;

    if(this.prim == null){//Se lista estiver vazia, cria novo no e adiciona no
    prim (primeiro na lista)
        this.prim = this.ult=novoNo;

    }else{// Enquanto não é o ultimo da lista e é maior que o atual, vai para o
    próximo

        while (atual != null && comparador.compare(novoValor, atual.getValor())
    >= 0) { // Enquanto atual não for null e novoValor for maior ou igual que
    atual.getValor
            ant = atual;// No atual vira o anterior
            atual = atual.getProx();// Atual passa apontar para o próximo No
        }

        if(ant == null){// No for menor que o primeiro No da lista, logo tem que
    ser o novo primeiro No
            novoNo.setProx(this.prim);
            this.prim = novoNo;

        } else if (atual == null ) { // Chegou ao final da lista, No vai ser
    inserido na última posição

```

```

        this.ult.setProx(novoNo);
        this.ult = novoNo;

    } else{
        ant.setProx(novoNo);
        novoNo.setProx(atual);
    }

}
this.quant++;
return true;
}

```

Caso seja ordenada, ele vai utilizar comparadores para dizer se deve adicionar o novo elemento antes ou depois de algum dos elementos. E como em uma lista ordenada, o elemento que for adicionado terá que percorrer a lista para ser identificado onde ele vai ser colocado. No pior caso ele vai ser $O(n)$, ou seja, no pior caso, onde ele vai ter que percorrer a lista toda para depois perceber que será colocado no final, ele vai ter percorrido n vezes dentro do while. Sendo n o tamanho total da lista.

Linha	Instrução	Custo unitário	Repetições (pior caso)
45 - 52	Adicionar	$O(1)$	$O(1)$
54 - 67	inserirElementoNaoOrd	$O(1)$	$O(1)$
69 - 104	inserirElementoOrd	$O(1)$	$O(n)$

Ordem de complexidade no pior caso: $O(n)$

2.2 pesquisar

```

107 //-----Pesquisar
108 @Override 5 usages João Victor S. Araujo +1
109 public T pesquisar(T valor) {
110     return pesquisar(valor, this.comparador);
111 }
112
113 // Permite buscar por outro critério (nesse caso, nome), mais fácil de implementar no menu
114 public T pesquisar(T valor, Comparator<T> criterio) { 3 usages João Victor S. Araujo
115     if (criterio == null) {
116         throw new IllegalArgumentException("Um comparador válido é necessário para a pesquisa.");
117     }
118
119     No<T> atual = this.prim;
120
121     while (atual != null) {
122         int cmp = criterio.compare(valor, atual.getValor());
123
124         if (cmp == 0) {
125             return atual.getValor();
126         }
127
128         // Se a lista tiver ordenada pelo critério passado, para de procurar caso passe da posição onde deveria estar.
129         // Adicionado "this.comparador != null" para evitar NPE no .getClass() caso a lista não tenha comparador base.
130         if (this.ehOrdenada && this.comparador != null && criterio.getClass() == this.comparador.getClass() && cmp < 0) {
131             break;
132         }
133
134         atual = atual.getProx();
135     }
136
137     return null;
138 }

```

Figura 2.2.1

```

public T pesquisar(T valor) {
    return pesquisar(valor, this.comparador);
}

// Permite buscar por outro critério (nesse caso, nome), mais fácil de
implementar no menu
public T pesquisar(T valor, Comparator<T> criterio) {
    if (criterio == null) {
        throw new IllegalArgumentException("Um comparador válido é necessário
para a pesquisa.");
    }

    No<T> atual = this.prim;

    while (atual != null) {
        int cmp = criterio.compare(valor, atual.getValor());

        if (cmp == 0) {
            return atual.getValor();
        }

        // Se a lista tiver ordenada pelo critério passado, para de procurar caso
passe da posição onde deveria estar.
        // Adicionado "this.comparador != null" para evitar NPE no .getClass()
caso a lista não tenha comparador base.
        if (this.ehOrdenada && this.comparador != null && criterio.getClass() ==

```

```

    this.comparador.getClass() && cmp < 0) {
        break;
    }

    atual = atual.getProx();
}

return null;
}

```

No método pesquisar, iremos cair em uma situação parecida, pois, ao pesquisar, no melhor caso o elemento que está procurando vai ser encontrado no primeiro endereço que o método pesquisar, sendo assim $O(1)$. Mas, no pior caso, o método pesquisar irá se repetir n vezes, pois o elemento procurado vai estar no último endereço a ser checado da lista, virando assim uma complexidade $O(n)$.

```

140 public ListaEncadeada<T> pesquisarTodosporNome(T nome, Comparator<T> Criterio){// Metodo para identificar multiplos usuarios com o mesmo nome 1 usage 1 valor *
141
142     ListaEncadeada<T> ListaNomesIguas = new ListaEncadeada<T>();
143     No<T> atual = prim;
144
145     while (atual != null) {
146
147         T elemento = atual.getValor();
148
149         if (Criterio.compare(nome, atual.getValor()) == 0) {
150             ListaNomesIguas.adicionar(elemento);
151         }
152
153         atual = atual.getProx();
154     }
155
156     return ListaNomesIguas;
157 }

```

Figura 2.2.2

```

public ListaEncadeada<T> pesquisarTodosporNome(T nome, Comparator<T>
Criterio){// Metodo para identificar multiplos usuarios com o mesmo nome

    ListaEncadeada<T> ListaNomesIguas = new ListaEncadeada<T>();
    No<T> atual = prim;

    while (atual != null) {

        T elemento = atual.getValor();

        if (Criterio.compare(nome, atual.getValor()) == 0) {
            ListaNomesIguas.adicionar(elemento);
        }

        atual = atual.getProx();
    }

    return ListaNomesIguas;
}

```

Já o método de *pesquisarTodosporNome*, não estará procurando por um elemento. Estará procurando por todos os elementos que compartilham uma semelhança com ele. Ou seja, se estivermos procurando todos os alunos com o nome "Victorio", o método vai andar n vezes até encontrar um elemento e vai registrar que o encontrou. Se tornando assim $O(n)$ também, uma vez que ele não vai precisar recomençar a busca, já que, toda vez que encontrar um elemento, ele registra. A diferença dele para o método de pesquisa normal, é que ele será sempre $O(n)$, pois, diferente do método de pesquisa normal, ele vai ter que checar a lista toda para garantir que não existe mais nenhum "Victorio" na lista.

Linha	Instrução	Custo unitário	Repetições (pior caso)
108 - 138	Pesquisar	$O(1)$	$O(n)$
140 - 157	<i>pesquisaTodosporNome</i>	$O(n)$	$O(n)$

Ordem de complexidade no pior caso: $O(n)$

2.3 remover

```
159 //-----Remover
160 @Override 2 usages 2 João Victor S. Araujo +1
161 public boolean remover(T valor) {
162     return remover(valor, this.comparador);
163 }
164
165 // Permite remover por outro critério (mesmo padrão usado no pesquisar)
166 public boolean remover(T valor, Comparator<T> criterio) { 2 usages 2 João Victor S. Araujo +1
167     No<T> atual, ant;
168
169     atual = this.prim;
170     ant = null;
171
172     while (atual != null) {
173         int cmp = criterio.compare(valor, atual.getValor());
174
175         if (cmp == 0) {
176             if (ant == null) { // Removendo o primeiro elemento da lista
177                 this.prim = atual.getProx();
178                 if (this.prim == null) {
179                     this.ult = null;
180                 }
181             } else { // Removendo do meio ou do fim
182                 ant.setProx(atual.getProx());
183                 if (atual == this.ult) {
184                     this.ult = ant;
185                 }
186             }
187
188             this.quant--;
189             return true;
190         }
191
192         // Mesmo padrão e verificação utilizados no metodo remover,
193         // para de procurar caso passe da posição que deveria estar.
194         if (this.ehOrdenada && criterio.getClass() == this.comparador.getClass() && cmp < 0) {
195             break;
196         }
197
198         ant = atual;
199         atual = atual.getProx();
200     }
201
202     return false;
203 }
```

Figura 2.3.1

```
// _____ Remove
@Override
public boolean remover(T valor) {
    return remover(valor, this.comparador);
}

// Permite remover por outro critério (mesmo padrão usado no pesquisar)
public boolean remover(T valor, Comparator<T> criterio) {
    No<T> atual, ant;

    atual = this.prim;
    ant = null;

    while (atual != null) {
        int cmp = criterio.compare(valor, atual.getValor());

        if (cmp == 0) {
            if (ant == null) { // Removendo o primeiro elemento da lista
                this.prim = atual.getProx();
                if (this.prim == null) {
                    this.ult = null;
                }
            } else { // Removendo do meio ou do fim
                ant.setProx(atual.getProx());
                if (atual == this.ult) {
                    this.ult = ant;
                }
            }

            this.quant--;
            return true;
        }

        // Mesmo padrão e verificação utilizados no metodo remover,
        // para de procurar caso passe da posição que deveria estar.
        if (this.ehOrdenada && criterio.getClass() == this.comparador.getClass()
            && cmp < 0) {
            break;
        }

        ant = atual;
        atual = atual.getProx();
    }

    return false;
}
```

No método *remove* será no pior caso $O(n)$, quando o elemento que queremos eliminar esteja na posição n da lista. Já seu melhor caso é o de complexidade $O(1)$, onde o elemento a ser eliminado se encontra no primeiro endereço da lista. Isso por utilizar o método buscar ($O(n)$ no pior caso), além de somar o custo operacional de remoção que é $O(1)$ sempre (constante).

Linha	Instrução	Custo unitário	Repetições (pior caso)
162 - 205	remove	$O(1)$	$O(n)$

Ordem de complexidade no pior caso: $O(n)$

2.4 quantidadeNos

```

207 //-----Quantidade de Nos total
208 @Override 2 usages  Kaiva S Gums +1
209 public int quantidadeNos() {
210     return quant;
211 }
```

Figura 2.4.1

```

//-----Quantidade de Nos total
@Override
public int quantidadeNos() {
    return quant;
}
```

O método devolve um contador mantido a cada inserção e remoção. Contudo há duas opções de executar o método que implicam em custos computacionais estruturalmente distintos: 1) como está implementado com um contador ou 2) percorrer a lista somando a uma variável contadora que é criada e descartada a cada chamada da função *quantidadeNos()*. Para a opção 1, adiciona-se um breve overhead para toda inserção/remoção de nós na lista, o que significa dizer que se acrescenta uma constante para tais casos, e acionar a função significa também execução constante ($O(1)$). Ao passo que, para a opção 2, haverá somente o trabalho de percorrer a lista inteira para toda chamada da função, sendo assim $O(n)$.

Linha	Instrução	Custo unitário	Repetições (pior caso)
208 - 211	quantidadeNos	$O(1)$	$O(1)$

Ordem de complexidade no pior caso: $O(1)$

2.5 Listas ordenadas e não-ordenadas

Método	Lista não-ordenada	Lista ordenada
adicionar	$O(1)$	$O(n)$
pesquisar	$O(n)$	$O(n)$
remover	$O(n)$	$O(n)$
quantidadeNos	$O(1)$	$O(1)$

A diferença mais notável entre a lista ordenada e a não-ordenada se observa para os métodos de adicionar e buscar. Para o método adicionar, a ordenada, certamente, apresentará custo adicional para todo caso comparado com a não-ordenada; exatamente, por ter de manter a condição de ordenada e, com isso, ter de percorrer até achar lugar onde inserir o novo nó. Contudo, a lista ordenada pode demonstrar vantagem para o método de pesquisa, quando é um vetor que se utiliza de pesquisa binária recursiva (Recursive Binary Search) que se baseia da seguinte forma: divide-se a lista em 3 porções, sendo a do meio um único item pivô; não sendo o pivô, se o item é menor que o pivô, chama-se a função novamente sobre o trecho de lista menor que o pivô; caso contrário, chama-se para o trecho de lista maior que o pivô; reduzindo a complexidade da busca de $O(n)$, em que se percorre a lista inteira, para $O(\log_2 n)$. Entretanto, nós utilizamos a estrutura de lista encadeada e não vetor, o que torna tal tipo de busca inviável. Isso se justifica por: em vetores, o acesso é $O(1)$ por estar em memória contígua, ao passo que tomar o item ao meio da lista do tipo encadeada requer percorrer a lista a partir de sua origem (ou fim, se duplamente encadeada).

3. Análise empírica de complexidade

3.1 Ambiente e metodologia

Item	Descrição
Processador	i5-10400F 2.90GHz
Memória	16 GB DDR4
Sistema operacional	Windows 11
Versão da JVM	25.0.1
Modo de medição	System.nanoTime()
Repetições por medição	11

3.2 Tempo de montagem da lista

n (registros)	Não-ordenada (ms)	Ordenada (ms)	Razão ord./não-ord.	Observações
100	3.00265	0.00164545	0,0005x	primeira rodada de testes, valor pouco confiável
25.000	38.1701	3688.1945	96,6x	
50.000	24.4321	11015.18835	450,8x	
100.000	24.89455	137096.4896	5507,1x	maior salto da série
200.000	85.58655	755219.5104	8824,0x	



Figura 3.2.1

O tempo gasto para a inserção de uma lista ordenada cresce substancialmente mais rápido que a não-ordenada, pelo seguinte motivo: ela deve buscar a posição em que o item não é nem menor que o anterior, nem maior que o sucessor, utilizando a função buscar que possui complexidade $O(n)$, além da função adicionar que é utilizada para ambos os casos de todo modo (o que não é o fator motivo para a discrepância entre os casos).

3.3 Tempo de pesquisa pelo último elemento (por matrícula)

n (registros)	Não-ordenada (ms)	Ordenada (ms)	Razão ord./não-ord.	Observações
100	0.01925	0.11055	5,74x	
25.000	1.4942	1.3754	0,92x	
50.000	1.65675	0.4751	0,29x	ordenada mais rápida que a não-ordenada
100.000	4.62305	3.55825	0,77x	
200.000	5.3714	8.2139	1,53x	

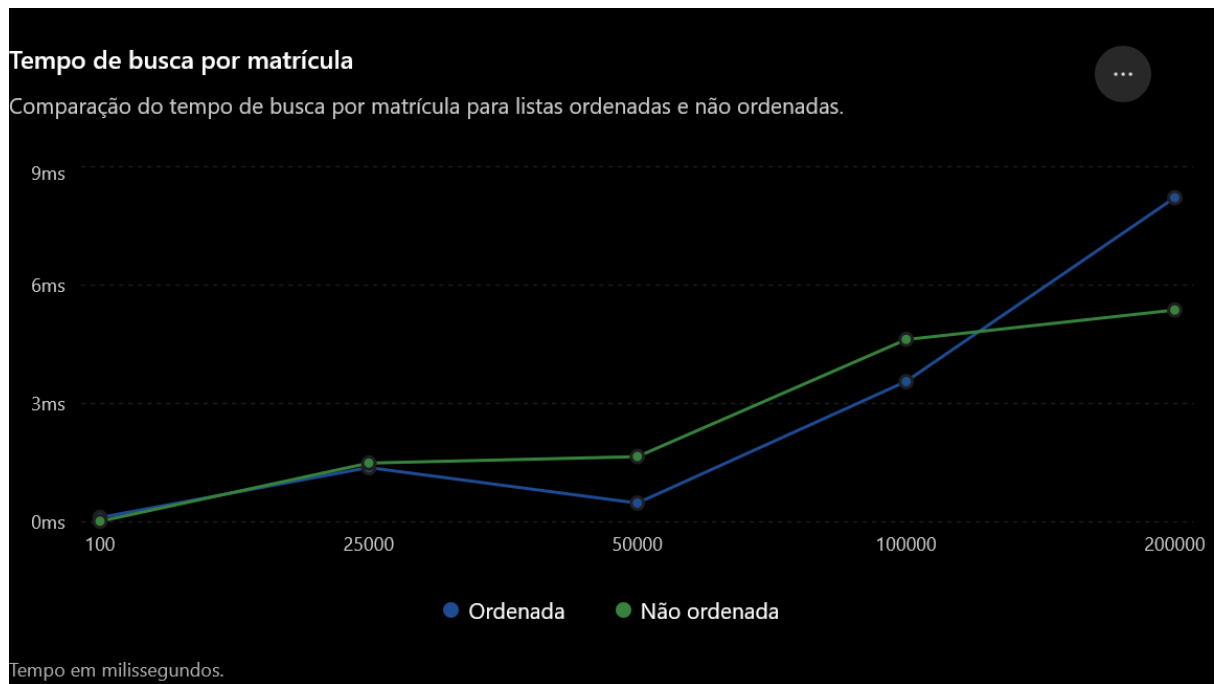


Figura 3.3.1

Como não fizemos o uso da busca por divisão recursiva por 2, por não termos estruturado a lista em vetor; os resultados de ambos os casos tendem a ser dependentes somente do acaso de convir do elemento estar mais próximo. Caso tivéssemos implementado a divisão recursiva por 2 em lista ordenada e do tipo vetor, o método buscar() para a lista ordenada haveria de apresentar menor complexidade, pois $O(\log_2 n) < O(n)$.

3.4 Tempo de pesquisa pelo último elemento (por nome)

n (registros)	Não-ordenada (ms)	Ordenada (ms)	Razão ord./não-ord.	Observações
100	0.00375	0.0135	3,60x	
25.000	0.0041	0.01015	2,48x	
50.000	0.0004	0.0003	0,75x	tempo cai bruscamente em ambas as listas
100.000	0.0003	0.0003	1,00x	tempo estabiliza
200.000	0.0003	0.0003	1,00x	tempo estabiliza

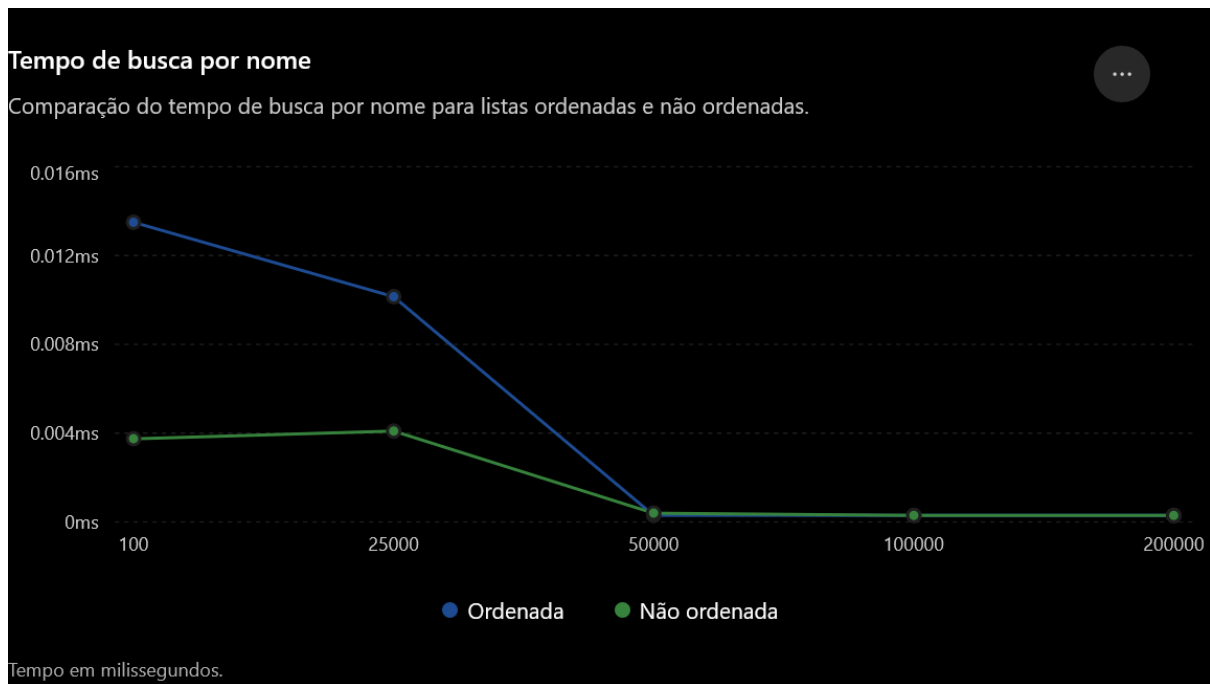


Figura 3.4.1

A busca por nome apresenta um comportamento particularmente interessante nos resultados obtidos. Isso ocorre devido à forma como o método `pesquisar()` está implementado: ele percorre a lista e retorna assim que encontra o primeiro elemento cujo nome seja igual ao nome procurado.

No experimento, o nome utilizado como chave de busca é o nome do último aluno da lista. Entretanto, o algoritmo não possui nenhuma informação adicional que permita determinar se o aluno encontrado é realmente esse último aluno. Dessa forma, caso existam outros alunos com o mesmo nome anteriormente na lista, a busca será encerrada assim que a primeira ocorrência for encontrada.

Por exemplo, se o nome do último aluno for "Caio" e existir outro aluno chamado "Caio" anteriormente na lista, o algoritmo encontrará esse primeiro "Caio" e encerrará a busca, sem chegar ao aluno que ocupa o último nó.

Isso também ajuda a explicar por que os tempos de busca por nome tendem a ser menores nos arquivos maiores. Em conjuntos de dados maiores, aumenta a probabilidade de existirem nomes repetidos. Consequentemente, aumenta a possibilidade de o nome do último aluno aparecer anteriormente na lista, fazendo com que a busca seja encerrada rapidamente.

Já em conjuntos menores, há uma menor probabilidade de repetição do nome procurado. Assim, o nome do último aluno possui maior chance de ser encontrado somente próximo ao final da lista, fazendo com que uma quantidade maior de nós seja percorrida.

Portanto, os tempos extremamente baixos observados nos arquivos maiores não significam que a busca por nome possui complexidade $O(1)$. Eles são consequência do critério de parada utilizado pelo algoritmo e da distribuição dos nomes nos dados de entrada. No pior caso, quando o nome procurado aparece somente no último nó, a busca percorre toda a lista, mantendo complexidade $O(n)$.

3.5 Tempo de remoção do último elemento

n (registros)	Não-ordenada (ms)	Ordenada (ms)	Razão ord./não-ord.	Observações
100	0.0325	0.1281	3,94x	
25.000	2.3191	27.6584	11,93x	pico atípico na lista ordenada
50.000	2.78685	3.3008	1,18x	
100.000	4.83565	3.6025	0,75x	
200.000	5.333	8.3141	1,56x	



Figura 3.5.1

O tempo de remoção se comporta de maneira a demonstrar complexidade $O(n)$, uma vez que ambos utilizam a função buscar, indistinta para ambos os tipos de lista, e executam a própria remoção, também igual para ambos os casos. Para o caso de termos implementado o algoritmo de busca por divisão binária recursiva em vetor ordenado, a lista ordenada demonstraria menor custo computacional pelo caso já descrito anteriormente.

Há, contudo, uma acentuada anomalia para o caso de 25 000 nós. A causa provável é discutida na seção 3.6.

3.6 Discussão

Na montagem, a lista não-ordenada permanece entre 24 e 86 ms nas quatro escalas, o tempo é dominado pela leitura do arquivo, não pela inserção, enquanto a ordenada salta de 3,7 para 755 segundos, comportamento quadrático. As razões entre escalas sucessivas, contudo, não são os 4x esperados ao dobrar n : obtivemos 2,99x, 12,45x e 5,51x. O desvio não é algorítmico. Como as matrículas vêm em ordem crescente no arquivo, toda inserção cai no pior caso e a montagem executa $n^2/2$ comparações; convertendo os tempos, passamos de 114 milhões de comparações por segundo com 50.000 registros para 26 milhões com 200.000. A mesma máquina tornou-se quatro vezes mais lenta por comparação. A causa está na memória: cada registro ocupa por volta de 120 bytes entre o nó, o Aluno e a String, de modo que a lista ocupa cerca de 6 MB com 50.000 registros e 24 MB com 200.000, contra 12 MB de cache L3 do processador. Até 50.000 a lista cabe no cache; a partir de 100.000, cada salto de ponteiro exige acesso à memória principal. A ordem permanece quadrática; cresce a constante que a notação $O(n)$ descarta.

A pesquisa por matrícula cresce com n nós dois regimes, como previsto, com dispersão compatível com a escala: uma varredura de 100.000 nós custa por volta de um milissegundo, e nessa faixa a compilação JIT e o escalonamento do sistema pesam tanto quanto o tamanho da lista. Os dois regimes empatam porque a parada antecipada não ajuda quando o alvo é o último elemento. A remoção acompanha a busca 5,3714 ms contra 5,3330 ms com 200.000 registros, o que confirma que o religamento de ponteiros não acrescenta termo dependente de n ; a anomalia dos 25.000 nós ordenados é a primeira remoção de grande porte da bateria, ainda sob compilação JIT, e não se repete nas escalas maiores.

A pesquisa por nome foi a única a destoar da previsão, e a causa está nos dados, não no método. O gerador produz apenas dez nomes distintos, repetidos a cada dez registros; como o alvo é o nome do último nó e a busca encerra na primeira ocorrência, ela acerta no primeiro nó da lista não-ordenada e na décima posição da ordenada. O número de comparações independe de n , daí o tempo constante. A coluna mede, portanto, um acerto no início da lista, e não o pior caso que permanece $O(n)$, como a busca por matrícula, de chave única, comprova.

Em suma, as fontes de desvio, em ordem de impacto: a configuração do arquivo de entrada, a localidade de memória, a compilação JIT, a coleta de lixo e, por fim, a leitura de disco e o ruído do sistema operacional. Foi para conter as duas últimas que descartamos a primeira das cinco repetições e adotamos a mediana de onze medições por teste.

4. Considerações finais

A lista encadeada favorece a inserção sem ordenação, de custo constante; o uso de memória proporcional ao que está guardado, sem realocação; e a simplicidade de uma implementação genérica, em que a troca de domínio não exige alterar uma linha da ListaEncadeada, apenas fornecer o Comparador adequado.

Penaliza, em contrapartida, todo acesso não sequencial. Alcançar o k -ésimo elemento exige percorrer os $k-1$ anteriores, e daí decorre a limitação que os dados evidenciam: toda busca é linear, e manter a lista ordenada não altera essa ordem. A ordenação encarece a inserção de $O(1)$ para $O(n)$ sem reduzir a ordem da busca, pois não há como saltar ao elemento do meio sem percorrer até ele. Seu único ganho é a parada antecipada na busca mal sucedida, que reduz o caso médio, mas preserva o pior caso em $O(n)$. A estrutura é adequada, portanto, onde predominam inserções e percursos sequenciais e a consulta por chave é rara: filas, pilhas, acumulação de registros lidos depois de uma vez.

Se o requisito fosse busca frequente por chave, escolheríamos outra estrutura: uma árvore binária de busca balanceada, caso fosse preciso percorrer os dados em ordem, pela ramificação que descarta metade dos elementos a cada comparação; uma tabela de dispersão, caso não fosse; ou um vetor ordenado com busca binária, se o conjunto fosse estático. Fica a lição de que a análise assintótica prevê a forma da curva, não o valor de cada ponto ela antecipou o crescimento quadrático da montagem ordenada, mas nada dizia sobre a queda de rendimento entre 50.000 e 100.000 registros, que só o tamanho do cache explica.

Referências

- CORMEN, T. H. et al. Algoritmos: teoria e prática. Rio de Janeiro: LTC. (capítulo de análise de algoritmos)
- W3SCHOOLS. *Java String format() method*. Disponível em: https://www.w3schools.com/java/ref_string_format.asp. Acesso em: 09 set. 2026.
- STACK OVERFLOW. *How to choose file in Java*. Disponível em: <https://stackoverflow.com/questions/40255039/how-to-choose-file-in-java>. Acesso em: 09 set. 2026.
- CARVALHO, Victorio Albani de. *Implementação de Listas Encadeadas em Java (Object, Generics, Comparable e Comparator)*. Instituto Federal do Espírito Santo. Slides. Acesso em: 07 set. 2026.